

PLAN DE PRUEBAS DE SOFTWARE

Sistema de Cotizaciones FastKote

Cliente: Negocio de Catering “Chichi Está de Fiesta”

Asignatura: Análisis y Diseño de Software (ADSW)

NRC: 30723

Grupo de Trabajo: Grupo 7

Integrantes: Diana Guerra

Paul Moreno

Leonel Tipan

Semestre: Sexto Semestre

Fecha: 8 de julio de 2026

Índice

1. Introducción y Alcance de las Pruebas	2
1.1. Alcance Incluido (En Scope)	2
1.2. Alcance Excluido (Out of Scope)	2
2. Objetivos de Calidad	3
2.1. Objetivos Funcionales	3
2.2. Objetivos No Funcionales	3
3. Estrategia y Tipos de Prueba	4
4. Criterios de Entrada y Salida	5
4.1. Criterios de Entrada (Entry Criteria)	5
4.2. Criterios de Salida (Exit Criteria)	5
5. Recursos, Roles y Responsabilidades	6
5.1. Matriz RACI del Equipo (Grupo 7)	6
6. Cronograma y Planificación	7
7. Riesgos y Medidas de Mitigación	8
8. Diseño de Casos de Prueba (Especificación)	9
8.1. Casos de Prueba de Autenticación y Seguridad (RF1)	9
8.2. Casos de Prueba de Clientes y Consentimiento LOPDP (RF2 - RF3)	10
8.3. Casos de Prueba de Gestión de Cotizaciones y Precios (RF12 - RF13)	10
8.4. Casos de Prueba de Agenda y Cambios de Estado (RF14)	11
8.5. Casos de Prueba del Gateway de WhatsApp (RF15)	12
9. Gestión de Datos de Prueba	13
10. Herramientas y Entornos de Prueba	13
11. Estrategia de Automatización	13
12. Integración DevOps y CI/CD	14
13. Métricas de Seguimiento y Calidad	14

1 Introducción y Alcance de las Pruebas

El presente documento define el plan de pruebas exhaustivo para el sistema web **FastKote**, diseñado para automatizar y optimizar la generación y el seguimiento de cotizaciones para el negocio de catering “*Chichi Está de Fiesta*”. Las pruebas descritas en este documento buscan asegurar que los cambios estructurales por capas y la implementación de patrones de diseño como *Mediator* y *Strategy* funcionen con los máximos estándares de calidad técnica y de negocio.

1.1 Alcance Incluido (En Scope)

El proceso de validación cubrirá los siguientes requisitos funcionales (RF) implementados en el prototipo:

- ✓ **RF1 (Seguridad y Control de Acceso):** Inicio de sesión seguro con JWT, validación de roles de usuario (*Admin* y *Empleado*), y el mecanismo de seguridad de bloqueo temporal tras 3 intentos fallidos (bloqueo por 5 minutos).
- ✓ **RF2 - RF4 (Gestión de Clientes):** Búsqueda avanzada de clientes (filtrado por nombre, identificación/RUC, correo, teléfono y estado), creación de clientes con obligatoriedad del consentimiento LOPDP (Ley Orgánica de Protección de Datos Personales), y edición mediante modales dinámicos.
- ✓ **RF5 - RF10 (Gestión de Personal):** Exclusivo para el rol *Admin*. Creación, modificación, asignación de roles mediante relación muchos a muchos (`employee_roles`), historial con consulta interactiva y desactivación lógica (estado *INACTIVE*).
- ✓ **RF11 - RF14 (Gestión de Cotizaciones y Agenda):** Filtros dinámicos de búsqueda. Wizard de creación en 4 pasos (Frontend) con aplicación de estrategias de cálculo del lado del Backend (*Strategy* de precios fijos, por número de niños y personalizada). Modal de cambio de estado de cotización con bloqueo automático de fechas en agenda para el estado *ACCEPTED* y liberación en *REJECTED* o *EXPIRED*.
- ✓ **RF15 (Exportación y Envío):** Generación física del PDF en memoria mediante `PDFKit` y llamada asíncrona a la pasarela desacoplada de WhatsApp (Green API, Ultramsg, o pasarela genérica) junto con su fallback de simulación controlada.

1.2 Alcance Excluido (Out of Scope)

Las siguientes características no forman parte de este plan de validación debido a los límites del alcance académico:

- × Integración con pasarelas de pago reales.
- × Costos reales de facturación por mensajes enviados mediante Green API o Ultramsg (se probará con entornos sandbox o simuladores).
- × Registro público directo de clientes (el botón “Registrarse” de la landing pública muestra un modal de construcción temporal).
- × El módulo de inventario real y control de movimientos (`inventory_movements`) que no afecte directamente a las cotizaciones base en este incremento.

2 Objetivos de Calidad

Para asegurar una implementación robusta y libre de errores antes de su paso a producción, se definen los siguientes objetivos medibles:

2.1 Objetivos Funcionales

1. **Precisión Matemática:** Validar que las estrategias de cálculo de cotización (`PerChildPricingStrategy`, `FixedPackagePricingStrategy` y `CustomPricingStrategy`) computen subtotales, descuentos, impuestos (tasa del 15 %) y totales con cero margen de error en el redondeo a dos decimales.
2. **Integridad Referencial de Datos:** Verificar que las inserciones y modificaciones respeten las restricciones de llave foránea de la base de datos PostgreSQL, evitando anomalías en la relación cotización-cliente o cotización-paquete.
3. **Control de Flujo de Negocio:** Validar que las cotizaciones en estado distinto a DRAFT queden bloqueadas para cualquier tipo de edición o modificación, según lo especificado en RF13.

2.2 Objetivos No Funcionales

1. **Seguridad de Autenticación:** Confirmar que las credenciales erróneas bloqueen la cuenta del usuario exactamente al tercer intento por un lapso estricto de 5 minutos, guardando el timestamp `lockedUntil` en la base de datos.
2. **Tiempo de Respuesta (Performance):** Asegurar que el tiempo de respuesta del backend para la generación de cotizaciones y del PDF no supere los 1.5 segundos en un entorno local de staging.
3. **Responsividad UX:** Probar que la interfaz de usuario en el frontend se adapte de forma fluida a dispositivos móviles (resoluciones comunes de smartphones y tablets) y navegadores modernos (Chrome, Firefox, Safari).

3 Estrategia y Tipos de Prueba

Nuestra estrategia de validación combina pruebas manuales y automatizadas aplicadas a lo largo de las capas del software (Presentación, Aplicación, Dominio, Infraestructura).

Cuadro 1: Tipos de pruebas a aplicar en FastKote

Tipo de Prueba	Descripción y Enfoque	Herramientas Sugeridas
Pruebas Unitarias	Validación aislada de las clases del patrón <i>Strategy</i> (cálculos) y <i>Mediator</i> .	Vitest / Jest
Pruebas de Integración	Comprobación del ciclo completo desde el controlador hasta la base de datos PostgreSQL usando Prisma en modo transaccional.	Supertest / Prisma Mock
Pruebas Funcionales E2E	Simulación de flujos de negocio completos desde la UI de React hasta la respuesta de la base de datos.	Playwright / Cypress
Pruebas de Seguridad	Intentos de elusión de roles JWT, inyecciones de SQL en inputs de búsqueda y pruebas de fuerza bruta en login.	OWASP ZAP
Pruebas de API	Testeo directo de los endpoints HTTP expuestos en el puerto 4000 de forma aislada.	Postman / Insomnia

4 Criterios de Entrada y Salida

4.1 Criterios de Entrada (Entry Criteria)

Para dar inicio a la fase formal de ejecución de pruebas, se deben cumplir las siguientes condiciones previas:

- **Disponibilidad del Entorno:** El contenedor Docker de PostgreSQL (`fastkote-postgres`) debe estar arriba y respondiendo en el puerto 5432.
- **Base de Datos Poblada:** Los scripts SQL `database/01_schema.sql` y `database/02_seed.sql` deben ejecutarse con éxito, sin reportar advertencias o errores sintácticos.
- **Variables de Entorno Configuradas:** El archivo `.env` del backend debe estar completo, definiendo el puerto (`PORT=4000`), la URI de PostgreSQL, y opcionalmente las variables de WhatsApp para pruebas de integración real (`WHATSAPP_API_URL` y `WHATSAPP_API_TOKEN`).
- **Generación de Artefactos de ORM:** Ejecutar `npx prisma generate` para asegurar que los tipos e índices mapeen correctamente con la base de datos Postgres y evitar el error recurrente de enums.
- **Estabilidad del Build:** Tanto el frontend en React como el backend en Express deben compilar y levantar de forma local sin errores críticos de transpiles de TypeScript.

4.2 Criterios de Salida (Exit Criteria)

La fase de pruebas de FastKote se considerará finalizada y apta para pase a producción cuando:

- **Cobertura de Casos:** Se haya ejecutado el 100 % de los casos de prueba diseñados e incluidos en la matriz de trazabilidad.
- **Cero Defectos Críticos:** No deben existir defectos abiertos de severidad “Bloqueante” o “Crítico” (por ejemplo, errores de cálculo financiero o brechas de control de acceso por rol).
- **Tasa de Aceptación:** Al menos el 95 % de los casos de prueba ejecutados deben culminar con estado “Exitoso”.
- **Aprobación de la Simulación:** El componente de envío de PDF y mensaje de WhatsApp (`WhatsAppGateway`) debe operar correctamente tanto en modo simulado como en la llamada REST al proveedor externo.

5 Recursos, Roles y Responsabilidades

La validación requiere una clara asignación de responsabilidades para evitar colisiones y asegurar la trazabilidad. Se define la siguiente estructura basada en los miembros reales del Grupo 7: Diana Guerra, Paul Moreno y Leonel Tipan, utilizando una matriz RACI (Responsable, Aprobador, Consultado, Informado).

5.1 Matriz RACI del Equipo (Grupo 7)

Cuadro 2: Matriz RACI con Miembros del Grupo 7

Actividad de Validación	Diana Guerra (QA Lead / PO)	Paul Moreno (Tester / DevOps)	Leonel Tipan (Desarrollador)
Análisis de Requisitos	A	R	C
Diseño de Casos de Prueba	A	R	C
Preparación de Datos (Seeds SQL)	C	R	R
Ejecución de Pruebas Manuales	A	R	I
Ejecución y Config. de CI/CD	C	R	A
Corrección de Defectos (Bugs)	I	I	R
Certificación de Calidad Final	A	R	I

6 Cronograma y Planificación

Las pruebas se integran dentro de los Sprints de desarrollo de forma iterativa bajo metodologías ágiles. A continuación se detalla la planificación temporal estructurada en un calendario compacto de exactamente ****7 días****, detallando las personas responsables de cada hito. Para evitar a toda costa el desborde vertical, la tabla se implementa mediante un entorno dinámico auto-paginable (**longtable**):

Cuadro 3: Calendario y Cronograma de Pruebas de 7 Días

Día	Fase	Detalle de Actividades	Entregable Clave	Responsables (Equipo)
Día 1	Fase 1: Preparación	Análisis de requisitos (casos de uso de cotización y login), diseño de casos de prueba y preparación de scripts de datos.	Casos de prueba detallados.	Diana Guerra, Paul Moreno
Día 2	Fase 2: Entornos	Despliegue de Docker PostgreSQL, ejecución de esquemas SQL y semillas, validación de variables de entorno del backend.	Base de datos con seeds cargada.	Paul Moreno
Día 3	Fase 3: Pruebas Unitarias y de API	Validación aislada de las clases <i>Strategy</i> (cálculos) y <i>Mediator</i> en backend, ejecución de pruebas de endpoints.	Reporte de cobertura unitaria.	Paul Moreno, Leonel Tipan
Día 4	Fase 4: Pruebas de Interfaz (UX)	Validación manual del login (seguridad de bloqueo de 3 intentos) y del wizard de cotizaciones en 4 pasos del frontend.	Registro preliminar de bugs en UI.	Paul Moreno
Día 5	Fase 5: Pruebas de Integración y WhatsApp	Envío y generación del PDF mediante PDFKit, pruebas de llamada a Green API / Ultramsg y simulación fallback.	Reporte de entrega de mensajería.	Paul Moreno, Leonel Tipan
Día 6	Fase 6: Regresión y Re-Testing	Corrección de bugs detectados por parte de desarrollo y re-ejecución del set de pruebas completo.	Log de incidentes resueltos.	Paul Moreno, Leonel Tipan
Día 7	Fase 7: Certificación y Cierre	Consolidación de métricas de calidad (densidad de defectos, PE), revisión con PO y firma de aceptación del sistema.	Acta de cierre de pruebas firmada.	Diana Guerra, Paul Moreno

7 Riesgos y Medidas de Mitigación

Cuadro 4: Matriz de riesgos y mitigación del proceso de pruebas

Riesgo Identificado	Impacto	Plan de Mitigación
El entorno de pruebas de WhatsApp externo (Green API/Ultrasmsg) cae o excede su cuota gratuita de peticiones.	Alto	Utilizar el mecanismo de simulación controlada implementado en WhatsAppGateway que se activa cuando no se definen las variables en el .env .
Inconsistencia en los Enums de PostgreSQL y Prisma ORM en tiempo de ejecución.	Medio	Incorporar una prueba pre-despliegue que fuerce npx prisma generate antes de levantar los servicios del backend.
Falta de datos transaccionales realistas para validar el cálculo dinámico de promociones.	Medio	Elaborar un archivo SQL robusto (seed.sql) con fechas de vigencia variables ajustadas dinámicamente (<i>relative timestamps</i>).
El equipo tiene poco tiempo para pruebas manuales de regresión.	Alto	Automatizar el set de pruebas básicas (Smoke Tests) del Login y la creación de cotizaciones utilizando scripts de Playwright .

8 Diseño de Casos de Prueba (Especificación)

Esta sección contiene el detalle de los casos de prueba clave diseñados para certificar los flujos de negocio más críticos del sistema FastKote.

8.1 Casos de Prueba de Autenticación y Seguridad (RF1)

Identificador	TC-RF01-01: Autenticación con Rol Válido (Administrador/Empleado)
Requisito Asociado	RF1: Autenticar Usuario y asignación de roles.
Objetivo	Verificar que un usuario con credenciales correctas acceda y reciba un token JWT con su rol respectivo.
Precondiciones	DB levantada. Usuario <code>admin</code> con clave <code>Admin123*</code> sembrado en base de datos.
Pasos de Ejecución	1. Ingresar a <code>http://localhost:5173/</code>. 2. Presionar el botón “Iniciar Sesión”. 3. Digitar usuario <code>admin</code> y contraseña <code>Admin123*</code>. 4. Hacer clic en “Entrar”.
Resultado Esperado	Redirección exitosa al panel interno (<code>/dashboard</code>). El menú lateral muestra las opciones de Empleados y Roles debido al rol de Administrador.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

Identificador	TC-RF01-02: Bloqueo de Cuenta por Intentos Fallidos
Requisito Asociado	RF1: Bloqueo por intentos fallidos.
Objetivo	Confirmar el bloqueo temporal de la cuenta por 5 minutos tras 3 intentos de login fallidos.
Precondiciones	Usuario <code>empleado</code> activo y desbloqueado en el sistema.
Pasos de Ejecución	1. Ingresar al formulario de login. 2. Intentar ingresar con usuario <code>empleado</code> y contraseña errónea <code>Falsa123</code> (Intento 1). 3. Reintentar con contraseña errónea <code>Falsa456</code> (Intento 2). 4. Reintentar con contraseña errónea <code>Falsa789</code> (Intento 3). 5. Observar el mensaje y realizar un 4to intento con la contraseña correcta <code>Empleado123*</code>.
Resultado Esperado	- En el tercer intento fallido, el backend devuelve código HTTP 401 con el mensaje “Usuario bloqueado por 5 minutos.” y actualiza <code>lockedUntil</code> en la base de datos. - El 4to intento con contraseña correcta sigue rechazándose debido al bloqueo temporal activo.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

8.2 Casos de Prueba de Clientes y Consentimiento LOPDP (RF2 - RF3)

Identificador	TC-RF03-01: Creación de Cliente Exitoso con Consentimiento LOPDP
Requisito Asociado	RF3: Crear Cliente con campos obligatorios y consentimiento LOPDP.
Objetivo	Validar el correcto registro de un nuevo cliente cuando se completan todos los campos obligatorios y se marca el check de privacidad.
Precondiciones	Empleado autenticado en la plataforma.
Pasos de Ejecución	1. Ir al módulo “Clientes” y hacer clic en “Nuevo Cliente”. 2. Llenar los campos: Tipo: NATURAL, Nombre: Juan Pérez, Cédula: 1712345678, Correo: juan@example.com, Teléfono: 0987654321, Dirección: Av. Amazonas. 3. Asegurar que la casilla “Acepto políticas de protección de datos (LOPDP)” esté seleccionada. 4. Presionar “Guardar”.
Resultado Esperado	Cliente creado con éxito. Se muestra en la lista. En base de datos, el registro tiene el campo <code>privacy_consent</code> en <code>true</code> .
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

8.3 Casos de Prueba de Gestión de Cotizaciones y Precios (RF12 - RF13)

Identificador	TC-RF12-01: Cotización con Paquete por Niño y Cantidad menor al Mínimo
Requisito Asociado	RF12: Wizard de Cotización y <i>Pricing Strategy</i> por niño.
Objetivo	Validar que si la cantidad de niños seleccionada en el wizard es inferior al mínimo del paquete catalogado, la estrategia de precios aplique la cantidad mínima establecida.
Precondiciones	Paquete “Fiesta Infantil Light” catalogado con <code>pricePerChild = 15.00</code> y <code>minChildren = 30</code> .
Pasos de Ejecución	1. Iniciar creación de cotización en el Wizard. 2. En el paso de selección de paquete, escoger “Fiesta Infantil Light”. 3. En cantidad de niños, ingresar 15 (valor menor al mínimo de 30). 4. Proceder al resumen financiero.
Resultado Esperado	La estrategia <code>PerChildPricingStrategy</code> calcula el subtotal usando el mínimo: $30 \times 15,00 = 450,00$ dólares (en lugar de $15 \times 15,00 = 225,00$). El desglose detalla el ajuste automático al mínimo.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

Identificador	TC-RF13-01: Bloqueo de Modificación de Cotización no Borrador
Requisito Asociado	RF13: Modificación permitida únicamente en estado borrador.
Objetivo	Asegurar que el sistema impida editar cualquier cotización cuyo estado haya cambiado a SENT, ACCEPTED o REJECTED.
Precondiciones	Existe una cotización con código COT-2026-001 en estado SENT.
Pasos de Ejecución	1. Buscar la cotización COT-2026-001 en el panel de cotizaciones. 2. Intentar hacer clic en el botón “Editar” o enviar una petición PUT/PATCH directa al endpoint de actualización: PUT /api/quotes/COT-2026-001 con nuevos datos.
Resultado Esperado	- Frontend: El botón de editar está deshabilitado u oculto para cotizaciones que no sean DRAFT. - Backend: La API retorna código HTTP 400 o 403 indicando que la cotización no está en estado borrador y no puede ser modificada.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

8.4 Casos de Prueba de Agenda y Cambios de Estado (RF14)

Identificador	TC-RF14-01: Bloqueo de Agenda tras Aceptación de Cotización
Requisito Asociado	RF14: Cambio de estado con estrategia de agenda.
Objetivo	Validar que al cambiar el estado de una cotización a ACCEPTED, se genere una reserva en la tabla <code>event_reservations</code> bloqueando la fecha del evento.
Precondiciones	Cotización COT-002 para el 2026-08-15 en estado SENT. No existen reservas previas en esa fecha.
Pasos de Ejecución	1. Abrir la cotización COT-002. 2. Hacer clic en “Cambiar Estado” y seleccionar “Aceptada” (ACCEPTED). 3. Guardar cambios. 4. Revisar la agenda/calendario y verificar la tabla <code>event_reservations</code> en la base de datos.
Resultado Esperado	- La cotización cambia a estado ACCEPTED. - Se inserta un registro en la base de datos con <code>event_date = '2026-08-15'</code> y <code>status = 'BLOCKED'</code> . - El calendario muestra el día ocupado bloqueando otras reservas para esa fecha.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

Identificador	TC-RF14-02: Liberación de Agenda tras Rechazo o Vencimiento
Requisito Asociado	RF14: Estrategia de cambio de estado y liberación.
Objetivo	Validar que al rechazar (REJECTED) o expirar una cotización previamente aceptada, la fecha se libere de la agenda (RELEASED).
Precondiciones	La fecha 2026-08-15 está bloqueada por la cotización COT-002.
Pasos de Ejecución	1. Abrir la cotización COT-002 (actualmente aceptada). 2. Cambiar el estado a “Rechazada” (REJECTED) debido a cancelación por parte del cliente. 3. Guardar cambios. 4. Revisar la agenda y la tabla <code>event_reservations</code>.
Resultado Esperado	- El estado de la cotización cambia a REJECTED. - La reserva asociada en <code>event_reservations</code> actualiza su estado a RELEASED. - La fecha queda libre y disponible para nuevas cotizaciones.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

8.5 Casos de Prueba del Gateway de WhatsApp (RF15)

Identificador	TC-RF15-01: Simulación de Envío de WhatsApp en Ausencia de Variables
Requisito Asociado	RF15: Gateway de WhatsApp desacoplado y simulación.
Objetivo	Verificar que el sistema no falle y continúe el flujo simulando la entrega si no se cuenta con credenciales API reales.
Precondiciones	Renombrar temporalmente las variables del backend: borrar <code>WHATSAPP_API_URL</code> y <code>WHATSAPP_API_TOKEN</code> del archivo <code>.env</code> . Reiniciar servidor.
Pasos de Ejecución	1. Generar o abrir una cotización existente. 2. Hacer clic en “Enviar por WhatsApp”.
Resultado Esperado	El backend detecta la falta de llaves y retorna un objeto de respuesta con <code>simulated = true</code> . La interfaz muestra una notificación flotante indicando: “ <i>Simulación: cotización enviada con éxito</i> ”.
Resultado Obtenido	[A ser llenado en ejecución]
Estado	Pendiente

9 Gestión de Datos de Prueba

Para que las pruebas sean consistentes y repetibles, se emplearán tres orígenes de datos:

1. **Datos Sembrados (Seeds SQL):** El archivo `database/02_seed.sql` introduce los registros fundamentales: roles primarios (**Admin**, **Empleado**), credenciales encriptadas de prueba, paquetes estándar (por ejemplo, “Paquete Boda Silver”, “Fiesta Infantil Premium”) e insumos básicos del catálogo.
2. **Datos de Entrada Sintéticos (Formularios):** Un juego de datos inválidos (cédulas de menos de 10 dígitos, correos con sintaxis inválida, cantidades de niños negativas) para probar los validadores de la capa de interfaz y esquemas Zod del backend.
3. **Variables Mock de Entorno:** Archivos de configuración locales que simulan respuestas exitosas y de fallo de Green API mediante servidores locales simulados con Express.

10 Herramientas y Entornos de Prueba

- **Entorno de Pruebas Local:**
 - Sistema Operativo: Linux Ubuntu 22.04 LTS o superior.
 - Base de Datos: PostgreSQL v15+ corriendo sobre Docker.
 - Frontend Server: Node.js (Vite dev server) en puerto 5173.
 - Backend Server: Node.js (Express con tsx-watch) en puerto 4000.
- **Herramientas para Reporte y Seguimiento:**
 - Control de versiones: Git / GitHub.
 - Gestión de casos e incidencias: Tablero Kanban (archivo XLSX de control interno de errores).

11 Estrategia de Automatización

Aunque el prototipo se valida inicialmente de forma manual por el equipo de QA, la estructura modular basada en arquitectura por capas permite una automatización directa y de bajo costo de mantenimiento en fases subsiguientes:

- **Pruebas de la Capa de Dominio y Aplicación:** Se recomienda automatizar las clases bajo `backend/src/application/quotes/strategies/pricing` mediante la herramienta Vitest. Dado que no tienen dependencias externas ni del ORM, su ejecución es ultrarrápida.
- **Pruebas de API (Integración):** Se sugiere configurar Supertest para invocar los endpoints de `/api/quotes` y `/api/clients` verificando códigos HTTP y payloads de respuesta de los esquemas de validación Zod.

12 Integración DevOps y CI/CD

Para integrar la verificación dentro del flujo continuo de entregas, se propone la creación de una tarea en GitHub Actions (`.github/workflows/test.yml`) que automatice las siguientes tareas ante cada *Pull Request* hacia la rama principal:

1. Levantar un servicio de base de datos PostgreSQL temporal (Docker service container).
2. Ejecutar las migraciones y sembrado de datos en la base de datos de test.
3. Ejecutar pruebas unitarias mediante el script `npm run test` (al configurarse).
4. Ejecutar el análisis estático de código (`eslint`) para validar calidad y estilo de codificación en TypeScript.

13 Métricas de Seguimiento y Calidad

Para cuantificar la evolución y eficacia del proceso de validación, el equipo utilizará los siguientes indicadores estándar:

- **Porcentaje de Ejecución de Pruebas:**

$$PE = \left(\frac{\text{Casos de Prueba Ejecutados}}{\text{Total de Casos Planificados}} \right) \times 100$$

- **Densidad de Defectos:** Número de fallos detectados por cada módulo del sistema (Clientes, Cotizaciones, Autenticación).
- **Tasa de Bloqueo:** Porcentaje de casos de prueba que no pudieron ejecutarse debido a bugs bloqueantes previos.
- **Tiempo Medio de Resolución de Bugs (MTTR):** Tiempo transcurrido desde que el tester reporta la incidencia hasta que desarrollo entrega la corrección validada.